

Reviewer Pack — Minimal Rank of Tropicalized Affine Schemes vs ACC⁰ Circuit ...

Entry aa3c5d9cf972 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 05:53:59 UTC

Minimal Rank of Tropicalized Affine Schemes vs ACC⁰ Circuit Size for AND-OR Trees

Entry ID: aa3c5d9cf972 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 05:53:59 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Complexity Theory: ACC⁰ Circuit Complexity for AND-Or Trees

Statement:

[‘For every explicit function f in P with a Boolean circuit of size s , there exists a tropicalized affine scheme A associated with the function such that the rank of A , denoted as $r(A)$, is bounded by a function of s : $r(A) = \Theta(s^{\{0.5\}})$.’, ‘Equivalently, for any AND-OR tree representing an explicit function f in P with depth d , there exists a corresponding tropicalized affine scheme whose rank $r(A)$ satisfies $r(A) \geq 2^{(d/2)}$.’]

Rationale (proposer’s reasoning):

[‘The study of tropical geometry can provide insights into the structure of computations by capturing the algebraic and geometric properties of functions. By associating tropicalized affine schemes with Boolean circuits, we may uncover a connection between the geometric complexity of these schemes and the computational complexity of the circuits they represent.’, ‘This conjecture bridges tropical geometry, which has not been commonly applied to ACC⁰ circuit complexity, with a specific complexity-theoretic object. If proven true, it would suggest that there is a significant geometric component in determining the size of AND-Or trees.’]

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 34139ea36e4f3b02

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman’s rank correlation coefficient between the ranks of tropicalized affine schemes and circuit sizes/depths exceeds 0.8 for at least 95% of the seeds, with no individual seed’s correlation falling below 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry ACC⁰ circuit complexity AND-OR trees - rank of tropicalized affine schemes AND ACC⁰ circuit size- $\theta(s^{\{0.5\}})$ bound for tropical geometry AND ACC⁰ depth d

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1207.1925v1] Introduction to tropical algebraic geometry - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/0810.5165v2] Some computations about Kazhdan-Lusztig cells in affine Weyl groups of rank 2 - [http://arxiv.org/abs/1807.06283v2] Tropical Fano Schemes - [http://arxiv.org/abs/0802.4323v1] Non-singular affine surfaces with self-maps - [http://arxiv.org/abs/2107.0663] Kinematic dependence of azimuthal anisotropies in p +Au, d +Au, ^3He +Au at $\sqrt{s_{NN}} = 200$ GeV - [http://arxiv.org/abs/2512.16347v3] GWTC-4.0: Searches for Gravitational-Wave Lensing Signatures

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_circuit(depth):
        if depth == 0:
            return random.choice([0, 1])
        else:
            left = generate_boolean_circuit(depth - 1)
            right = generate_boolean_circuit(depth - 1)
            return random.choice([left and right, left or right])

    def generate_tropicalized_affine_scheme(circuit):
        if isinstance(circuit, int):
            return [[circuit]]
        else:
            left = generate_tropicalized_affine_scheme(circuit[0])
            right = generate_tropicalized_affine_scheme(circuit[1])
```

```

        return [left[i] + right[i] for i in range(len(left))]

def rank(A):
    if not A or not A[0]:
        return 0
    m, n = len(A), len(A[0])
    for col in range(n):
        pivot_row = None
        for row in range(m):
            if A[row][col] != 0:
                pivot_row = row
                break
        if pivot_row is None:
            continue
        for i in range(m):
            if i != pivot_row:
                factor = A[i][col] / A[pivot_row][col]
                for j in range(n):
                    A[i][j] -= factor * A[pivot_row][j]
    return sum(1 for row in A if any(x != 0 for x in row))

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    circuit = generate_boolean_circuit(n)
    A = generate_tropicalized_affine_scheme(circuit)
    r_A = rank(A)
    results.append({
        "n": n,
        "circuit_size": len(circuit),
        "rank": r_A
    })

if not results:
    return {
        "metric_name": "Spearman's rank correlation",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No instances generated"
    }

def spearman_rank_correlation(x, y):
    n = len(x)
    ranks_x = {v: i for i, v in enumerate(sorted(set(x)), start=1)}
    ranks_y = {v: i for i, v in enumerate(sorted(set(y)), start=1)}
    d_squared_sum = sum((ranks_x[x[i]] - ranks_y[y[i]]) ** 2 for i in range(n))
    return 1 - (6 * d_squared_sum) / (n * (n**2 - 1))

x = [result["circuit_size"] for result in results]
y = [result["rank"] for result in results]
rho = spearman_rank_correlation(x, y)

return {
    "metric_name": "Spearman's rank correlation",

```

```

        "metric_value": rho,
        "instances_tested": len(results),
        "conjecture_holds": rho >= 0.8 and all(rho_i >= 0.5 for rho_i in [spearman_rank_correlation
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) if sys.argv[1:] else [2**i + 3**j for i in range(5) for j in range(5)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_rho = sum(result["metric_value"] for result in results if result["metric_value"] is not None) / len(results)
    std_rho = math.sqrt(sum((result["metric_value"] - mean_rho) ** 2 for result in results if result["metric_value"] is not None) / len(results))
    support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    if all(rho is not None and rho >= 0.8 for rho in [result["metric_value"] for result in results]):
        print(f"RESULT: SUPPORTED mean={mean_rho} std={std_rho} support_fraction={support_fraction}")
    elif any(result["conjecture_holds"] for result in results):
        first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"Spearman's rank correlation < 0.8\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=not_enough_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a8a3468c.py", line 103, in <module>
    trial_result = run_trial(seed)
                   ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a8a3468c.py", line 65, in run_trial
    "circuit_size": len(circuit),
                   ~~~~~
TypeError: object of type 'int' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test to ensure it completes successfully and produces the required data for correlation analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11899
2	preregistration	ollama_remote	glm4:latest	0	0	5995
3	novelty	ollama_remote	glm4:latest	0	0	4642
4	novelty	ollama_remote	glm4:latest	0	0	6307
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17561
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9303
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9217
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12618
9	judge	ollama_remote	glm4:latest	0	0	8450

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 85993 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/aa3c5d9cf972.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/aa3c5d9cf972.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/aa3c5d9cf972.tar.gz` (if generated)